

Лекция 7. Виды приложений Android

- Сервисы.
- Широковещательные намерения.
- Доступ к системным событиям.
- Обработка SMS, событий аккумулятора.
- Автозагрузка.
- Создание уведомлений.
- Архитектуры приложений, библиотеки Dagger 2, Koin.
- Живые обои. Виджеты рабочего стола. Геометрические фигуры из XML.

Сервисы

Сервисы в Android — это компонент приложения, предназначенный для выполнения длительных операций в фоновом режиме без участия пользовательского интерфейса. Они идеально подходят для задач, которые должны работать независимо от того, открыт ли экран приложения или нет. Примерами таких задач могут быть воспроизведение музыки, загрузка или синхронизация данных, обработка информации, отслеживание местоположения и другие фоновые процессы.

Пример:

```
class MusicService : Service() {
    private lateinit var mediaPlayer: MediaPlayer

    override fun onCreate() {
        super.onCreate()
        // Инициализация медиаплеера с бесконечным воспроизведением
        mediaPlayer = MediaPlayer.create(this, R.raw.background_music)
        mediaPlayer.isLooping = true
    }
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        // Запуск воспроизведения при старте сервиса
        mediaPlayer.start()
    }
}
```

```

        return START_STICKY // Система перезапустит сервис, если он был убит
    }
    override fun onBind(intent: Intent?): IBinder? = null

    override fun onDestroy() {
        super.onDestroy()
        // Остановка и освобождение ресурсов при завершении
        mediaPlayer.stop()
        mediaPlayer.release()
    }
}

```

В этом примере сервис MusicService реализует фоновое воспроизведение музыки. Метод onCreate() инициализирует MediaPlayer, устанавливая файл background_music из ресурсов и включая бесконечное воспроизведение. onStartCommand() запускает плеер при вызове startService(), а флаг START_STICKY гарантирует, что система перезапустит сервис, если он был остановлен из-за нехватки ресурсов. При уничтожении сервиса через stopService() вызывается onDestroy(), где плеер останавливается и освобождает ресурсы. Это предотвращает утечки памяти и обеспечивает корректное завершение работы.

Широковещательные намерения

Широковещательные намерения (BroadcastReceiver) позволяют реагировать на системные события, такие как изменение уровня заряда, получение SMS или подключение наушников. Например:

```

class BatteryReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context?, intent: Intent?) {
        // Извлечение данных о зарядке из намерения
        val level = intent?.getIntExtra(BatteryManager.EXTRA_LEVEL, 0)
        val scale = intent?.getIntExtra(BatteryManager.EXTRA_SCALE, 100)
        val batteryPct = level?.div(scale!!) * 100

        // Вывод уведомления при низком уровне заряда
        if (batteryPct < 20) {
            val channelId = "battery_low"
            val notification = Notification.Builder(context!!, channelId)
                .setContentTitle("Низкий уровень заряда")
                .setSmallIcon(R.drawable.ic_battery_alert)

```

```

        .setAutoCancel(true)

        val notificationManager =
            context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        notificationManager.notify(1, notification.build())
    }
}

```

Здесь `BatteryReceiver` обрабатывает системное событие `ACTION_BATTERY_CHANGED`, извлекая уровень заряда через `getIntExtra()`. Метод `onReceive()` вычисляет процент зарядки, сравнивает его с порогом (20%) и выводит уведомление через `Notification.Builder`. `NotificationManager` управляет отображением уведомления, а `notify()` отправляет его в статус-бар. Уведомление автоматически закрывается при нажатии (`setAutoCancel(true)`). Этот подход демонстрирует, как приложения могут реагировать на системные события и информировать пользователя даже в фоне.

Доступ к системным событиям

Для доступа к системным событиям в Android, таким как получение SMS, изменение уровня заряда батареи, подключение или отключение Wi-Fi, изменение языка системы и другие важные события, используется компонент приложения под названием `BroadcastReceiver` (приёмник широковещательных сообщений). Он позволяет приложению реагировать на такие события даже тогда, когда оно не запущено или находится в фоне.

`BroadcastReceiver` работает по принципу подписки: приложение объявляет интерес к определённым событиям, и система уведомляет его о наступлении этих событий с помощью `Intent` — специального объекта, передающего данные о произошедшем событии. Например, при получении SMS система отправляет широковещательное сообщение (`Broadcast`), и если в каком-либо приложении зарегистрирован `BroadcastReceiver`, слушающий это событие, он может обработать его — например, считать текст сообщения или выполнить другое действие.

Существует два способа регистрации `BroadcastReceiver`:

- Статическая регистрация (в манифесте) — BroadcastReceiver объявляется в файле AndroidManifest.xml, и он будет работать даже если приложение выключено. Однако начиная с Android 8 (API 26), большинство широковещательных намерений нельзя использовать этим способом из-за ограничений фоновой активности.
- Динамическая регистрация (в коде) — BroadcastReceiver регистрируется программно внутри Activity или Service и работает только пока приложение активно или пока явно не отменена регистрация. Этот способ более актуален в новых версиях Android.

Например, можно создать BroadcastReceiver, который будет реагировать на подключение устройства к электропитанию и запускать определённую логику, например, начинать синхронизацию данных при подключении к зарядке.

Следующий пример демонстрирует реализацию BroadcastReceiver, который перехватывает и обрабатывает входящие SMS-сообщения на устройстве под управлением Android:

```
class SmsReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context?, intent: Intent?) {
        // Проверка, что намерение содержит SMS
        if (intent?.action == "android.provider.Telephony.SMS_RECEIVED") {
            val bundle = intent.extras
            val messages = bundle?.get("pdus") as Array<*>?
            messages?.forEach { message ->
                // Преобразование PDU в SMS-сообщение
                val sms = SmsMessage.createFromPdu(message as ByteArray, "3gpp")
                Toast.makeText(context, "SMS от ${sms.displayOriginatingAddress}:
                ${sms.messageBody}", Toast.LENGTH_LONG).show()
            }
        }
    }
}
```

Здесь SmsReceiver перехватывает событие SMS_RECEIVED, извлекает из намерения массив pdus (протокол данных пользователя), преобразует его в SmsMessage и выводит информацию через Toast. Метод onReceive() работает в основном потоке, поэтому длительные операции должны быть переданы в

фоновые потоки. В Android 10+ требуется явное разрешение RECEIVE_SMS в манифесте и установка приложения как дефолтного SMS-менеджера, иначе сервис будет заблокирован системой.

Обработка SMS, событий аккумулятора

В Android приложения могут реагировать на различные системные события, такие как получение SMS или изменение состояния аккумулятора устройства. Для реакции на системные события, такие как получение SMS, необходимо зарегистрировать BroadcastReceiver динамически в активности.

Например:

```
class MainActivity : AppCompatActivity() {
    private val smsReceiver = SmsReceiver()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Регистрация ресивера для получения SMS
        val filter = IntentFilter("android.provider.Telephony.SMS_RECEIVED")
        registerReceiver(smsReceiver, filter)
    }
    override fun onDestroy() {
        super.onDestroy()
        // Отмена регистрации ресивера при выходе из активности
        unregisterReceiver(smsReceiver)
    }
}
```

Здесь MainActivity регистрирует SmsReceiver через IntentFilter, указывающий на событие SMS_RECEIVED. Это позволяет приложению обрабатывать SMS в реальном времени. При уничтожении активности unregisterReceiver() освобождает ресурсы, предотвращая утечки памяти. Такой подход применяется не только для SMS, но и для других системных событий, таких как ACTION_POWER_CONNECTED или ACTION_BATTERY_LOW.

Android также предоставляет возможность отслеживать изменения в состоянии аккумулятора устройства, такие как уровень заряда, состояние подключения к питанию (USB, AC, беспроводная зарядка), температура и т.

д. Эти данные полезны для энергоэффективных приложений, например, чтобы запускать фоновые задачи только при подключённом питании или предупреждать пользователя о низком заряде.

Системное событие, связанное с аккумулятором, генерируется с помощью Intent с действием "android.intent.action.BATTERY_CHANGED". Оно не требует регистрации фильтра через манифест или динамическую регистрацию, так как является статическим и отправляется сразу после запроса.

Пример получения информации об аккумуляторе:

```
val batteryIntent = context.registerReceiver(null,
IntentFilter(Intent.ACTION_BATTERY_CHANGED))
val level = batteryIntent?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1)
val scale = batteryIntent?.getIntExtra(BatteryManager.EXTRA_SCALE, -1)
val percent = level?.div(scale!!) * 100
val status = batteryIntent?.getIntExtra(BatteryManager.EXTRA_STATUS, -1)
val isCharging = status == BatteryManager.BATTERY_STATUS_CHARGING ||
    status == BatteryManager.BATTERY_STATUS_FULL
```

Также можно отслеживать другие параметры:

- BatteryManager.EXTRA_HEALTH — здоровье батареи
- BatteryManager.EXTRA_TEMPERATURE — температура (в десятых долях градуса Цельсия)
- BatteryManager.EXTRA_PLUGGED — способ подзарядки (USB, AC и т. д.)

Автозагрузка

Автозагрузка позволяет запускать сервисы или ресиверы после перезагрузки устройства, что критично для приложений, работающих в фоне. Следующий пример демонстрирует реализацию BroadcastReceiver, который реагирует на событие завершения загрузки устройства (перезагрузки) и запускает фоновый сервис.

```
class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context?, intent: Intent?) {
        // Проверка события перезагрузки
        if (intent?.action == Intent.ACTION_BOOT_COMPLETED) {
```

```

        val serviceIntent = Intent(context, MusicService::class.java)
        ContextCompat.startForegroundService(context!!, serviceIntent)
    }
}
}

```

То есть этот пример демонстрирует, как приложение может реагировать на перезагрузку устройства и автоматически запускать фоновый сервис, например, для продолжения воспроизведения музыки или других задач. Здесь `BootReceiver` обрабатывает событие `ACTION_BOOT_COMPLETED`, которое отправляется после загрузки системы. При срабатывании намерения `startForegroundService()` запускает `MusicService` в foreground-режиме, что необходимо для Android 8+ из-за ограничений на фоновые процессы. Для работы требуется разрешение `RECEIVE_BOOT_COMPLETED` в `AndroidManifest.xml`. Этот подход обеспечивает запуск сервиса даже после перезагрузки устройства, что полезно для долгосрочных задач.

Создание уведомлений

Уведомления — это важный элемент взаимодействия приложения с пользователем. Они позволяют информировать пользователя о событиях, происходящих в фоновом режиме, таких как получение SMS, изменение состояния аккумулятора, завершение длительной задачи или начало воспроизведения музыки после перезагрузки устройства.

В Android уведомления создаются с помощью `NotificationManager` и строятся с использованием `NotificationCompat.Builder` (из библиотеки `androidx.core:core`). Это позволяет создавать уведомления, совместимые с разными версиями Android, начиная с API 19 и выше.

Основные компоненты уведомления:

- Заголовок (Title) — краткое описание события.
- Содержание (Text) — основное сообщение уведомления.
- Иконка (Small Icon) — обязательный элемент, отображается рядом с уведомлением.

- Канал уведомлений (Notification Channel) — обязательный атрибут для Android 8 (API 26) и выше. Позволяет группировать уведомления по типу и управлять их настройками (звук, вибрация, важность).
- Действия (Actions) — кнопки под уведомлением, позволяющие пользователю быстро реагировать на событие (например, "Пауза", "Остановить").
- Приоритет / Важность (Importance) — определяет, насколько заметным будет уведомление (например, высокий уровень может вызвать баннер и звук).

Пример создания простого уведомления:

```
val channelId = "music_service_channel"
val notificationId = 1

// Создание канала уведомлений (для Android 8+)
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    val channel = NotificationChannel(channelId, "Music Service",
        NotificationManager.IMPORTANCE_DEFAULT)
    val manager = context.getSystemService(Context.NOTIFICATION_SERVICE) as
        NotificationManager
    manager.createNotificationChannel(channel)
}

// Создание уведомления
val notification = NotificationCompat.Builder(context, channelId)
    .setContentTitle("Музыкальный сервис")
    .setContentText("Воспроизведение музыки в фоне")
    .setSmallIcon(R.drawable.ic_music_note)
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .build()

// Отображение уведомления
val manager = context.getSystemService(Context.NOTIFICATION_SERVICE) as
    NotificationManager
manager.notify(notificationId, notification)
```

Архитектура приложений, библиотеки Dagger 2 и Koin

В современной разработке Android-приложений большое внимание уделяется архитектуре — организации кода таким образом, чтобы он был понятным, тестируемым, расширяемым и поддерживаемым. Для достижения этих целей используются архитектурные шаблоны (например, MVVM, MVP) и паттерны проектирования, такие как Dependency Injection (DI) — внедрение зависимостей.

Для реализации DI в Android существуют специализированные библиотеки, среди которых наиболее популярны Dagger 2 и Koin. Типичная архитектура Android-приложения включает следующие слои:

- View / UI Layer (Представление)

Содержит Activity, Fragment и другие элементы интерфейса.

Отвечает за отображение данных и взаимодействие с пользователем.

- ViewModel Layer

Предоставляет данные для UI и сохраняет их при изменении конфигурации (например, поворот экрана).

Используется в связке с LiveData или StateFlow.

- Repository Layer

Центральный слой, который предоставляет данные из разных источников: локальная БД (Room), сеть (Retrofit, OkHttp) и т. д.

Обеспечивает абстракцию доступа к данным.

- Data Layer

Содержит модели данных, DAO (Data Access Objects), API-интерфейсы и другие низкоуровневые компоненты.

Внедрение зависимостей — это подход, позволяющий объектам получать свои зависимости извне, а не создавать их самостоятельно. Это упрощает тестирование, делает код более гибким и модульным.

Следующий пример демонстрирует использование библиотеки Koin для реализации внедрения зависимостей (Dependency Injection) в Android-приложении.

```
// Пример с Koin: объявление модуля
val appModule = module {
    // Создание базы данных как single (один экземпляр на всё приложение)
    single {
        Room.databaseBuilder(get(), AppDatabase::class.java, "user-database").build()
    }

    // DAO как зависимость, предоставляемая через базу данных
    single { get<AppDatabase>().userDao() }
}

class MainActivity : AppCompatActivity() {
    // Внедрение зависимости через Koin
    private val userDao: UserDao by inject()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Использование DAO для получения данных
        val user = userDao.getUserById(1)
    }
}
```

Здесь модуль `appModule` описывает, как создавать объекты: `AppDatabase` через `Room` и `UserDao` из базы данных. В `MainActivity` `by inject()` автоматически внедряет `UserDao`, используя `Koin`. Это исключает ручное создание зависимостей и делает код чище. `Koin` подходит для простых приложений, тогда как `Dagger 2`, используя аннотации `@Inject` и `@Component`, работает быстрее за счёт генерации кода, но требует сложной настройки. Выбор между ними зависит от масштаба проекта: `Koin` — для скорости и простоты, `Dagger` — для производительности и контроля.

Живые обои. Виджеты рабочего стола. Геометрические фигуры из XML

В Android существует множество типов приложений и компонентов, которые позволяют создавать разнообразный пользовательский опыт — от стандартных приложений с интерфейсом до специализированных, таких как живые обои, виджеты рабочего стола и элементы интерфейса, построенные

на основе геометрических фигур в XML. Эти виды приложений или их частей расширяют функциональность и визуальное оформление устройства.

Живые обои — это специальный тип приложения, который заменяет статичное фоновое изображение динамически изменяющейся анимацией или интерактивной сценой. Такие приложения имеют следующие особенности:

- Реализуются через класс WallpaperService и внутренний класс Engine.
- Могут использовать OpenGL ES или Canvas для рисования.
- Работают в фоне, но не являются полноценными приложениями с UI.
- Поддерживают взаимодействие с пользователем (например, касания).

Следующий пример демонстрирует реализацию живых обоев в Android с использованием класса WallpaperService. Конкретно, здесь создается сервис LiveWallpaperService, который отвечает за отображение анимированного круга на экране устройства. Такие обои реагируют на изменения положения и могут динамически обновляться, создавая эффект движения или интерактивности.

```
class LiveWallpaperService : WallpaperService() {
    override fun onCreateEngine(): Engine {
        return object : Engine() {
            private lateinit var handler: Handler
            private var x = 0f
            private var y = 0f
            override fun onSurfaceCreated(holder: SurfaceHolder?) {
                super.onSurfaceCreated(holder)
                // Инициализация Handler для работы с потоками
                handler = Handler(Looper.getMainLooper())
            }
            override fun onDraw(holder: Canvas?) {
                holder?.let {
                    val paint = Paint().apply {
                        color = Color.RED
                        style = Paint.Style.FILL
                    }
                    holder.lockCanvas()?.let { canvas ->
                        canvas.drawCircle(x, y, 50f, paint)
                        holder.unlockCanvasAndPost(canvas)
                    }
                }
            }
        }
    }
}
```

```

    }
    override fun onOffsetsChanged(xOffset: Float, yOffset: Float, xStep: Float, yStep:
Float, xPixels: Int, yPixels: Int) {
        // Обновление позиции при смене ориентации
        this.x = xOffset * xPixels
        this.y = yOffset * yPixels
        invalidate() // Перерисовка
    }
}
}
}
}
}

```

В этом примере LiveWallpaperService наследуется от WallpaperService и реализует Engine для отрисовки. В onSurfaceCreated() создаётся Handler для работы с потоками. Метод onDraw() использует Canvas и Paint для рисования круга, а onOffsetsChanged() обновляет его позицию при смене ориентации. invalidate() вызывает перерисовку, гарантируя актуальность отображения. Этот пример демонстрирует, как создавать интерактивные живые обои, которые реагируют на изменения устройства и обеспечивают плавную анимацию.

Геометрические фигуры из XML

XML-разметка позволяет создавать графические элементы без программной отрисовки, используя предопределённые фигуры. Такие фигуры часто используются как фоновые элементы для кнопок, текстовых полей или других View.

Пример: Создание прямоугольника через XML-файл.

```

<!-- res/drawable/rectangle.xml -->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <!-- Цвет заливки -->
    <solid android:color="#FF0000" />
    <!-- Скругление углов -->
    <corners android:radius="16dp" />
    <!-- Обводка (необязательно) -->
    <stroke
        android:width="4dp"
        android:color="#000000" />

```

</shape>

Применение XML-фигуры в разметке:

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        // Использование XML-фигуры как фон для View  
        val customView = findViewById<View>(R.id.customView)  
        customView.setBackgroundResource(R.drawable.rectangle)  
    }  
}
```

XML-файлы в Android позволяют декларативно описывать геометрические фигуры без программной отрисовки. В примере `rectangle.xml` определяет прямоугольник с красной заливкой, скруглёнными углами и чёрной обводкой. Такую фигуру можно применить как фон для любого View через `setBackgroundResource()`, как показано в MainActivity. Это исключает необходимость работы с Canvas и Paint, упрощая реализацию. XML-файлы поддерживают типы фигур (`rectangle`, `oval`, `line`), стили (цвета, градиенты, радиусы) и анимации (через `animation-list`), делая их удобным инструментом для создания интерфейсных элементов. Такой подход особенно полезен для статичных фигур, так как изменения в XML-файле мгновенно отражаются в интерфейсе без перекомпиляции кода.